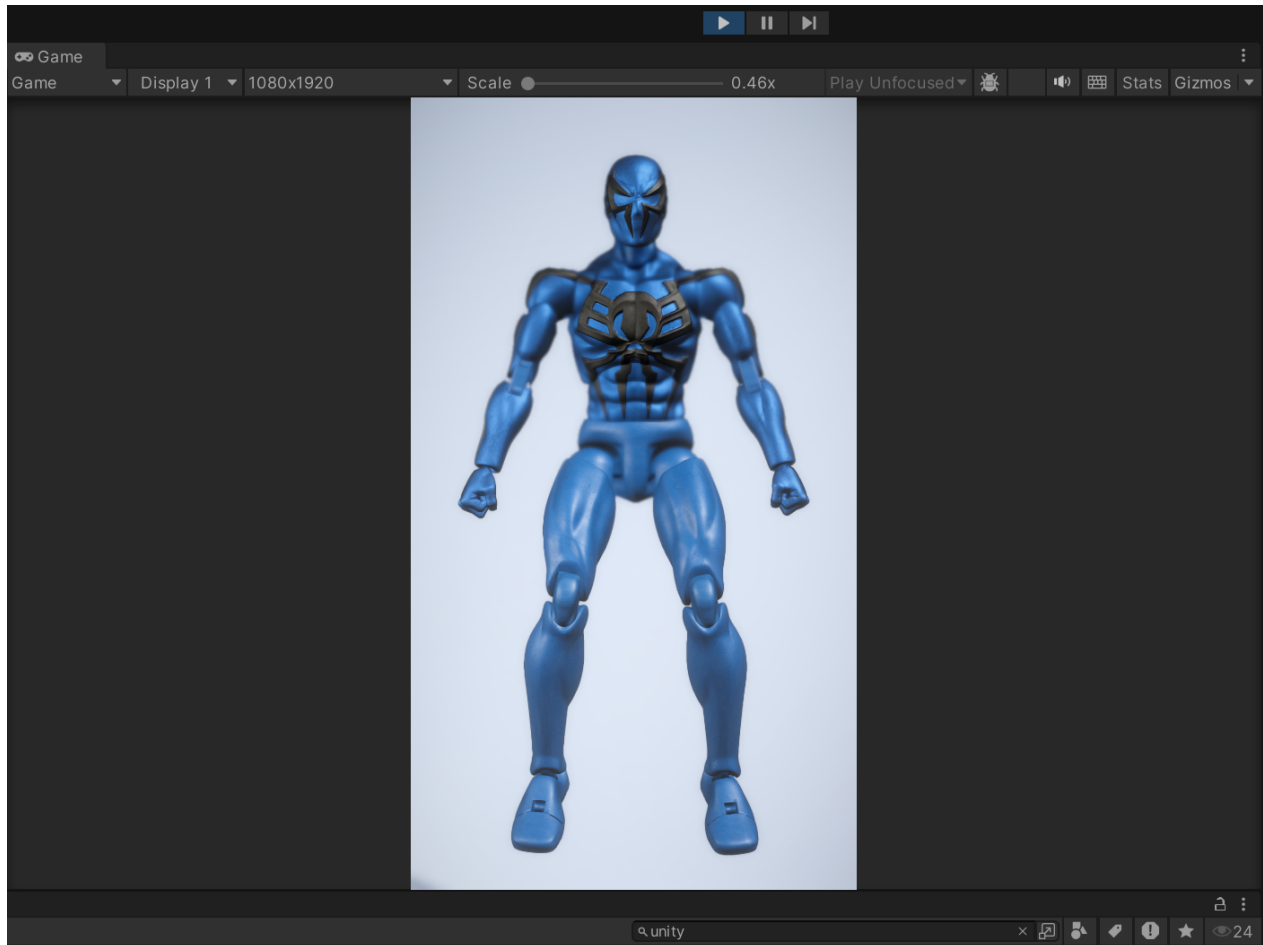


Animation Rigging in Unity

Make Your Robot Interactive. Prep It for AI.

A project-specific tutorial for Unity_Adam_Studio.unity



The robot in the portrait Game view. This screenshot is reused in context in Section 3.

How to use this guide

Follow the numbered procedures in order when opening or replacing the character. Sections labeled Project-confirmed come from serialized project files and supplied screenshots. Video-workflow sections adapt the referenced Brackeys demonstration without copying its wording. Unconfirmed items are called out explicitly.

PROJECT-CONFIRMED The analysis was read-only. No Unity scene, prefab, script, package, or project-setting file was changed.

Verified project facts

Item	Confirmed value
Unity Editor	2022.3.62f1 (revision 4af31df58517)
Project folder	FINAL_TOY_Mobile_V2 - WebGL
Current scene name	Assets/Scenes/Unity_Adam_Studio.unity (owner-confirmed rename)
Animation Rigging package	com.unity.animation.rigging 1.2.1
Render pipeline	Universal Render Pipeline 14.0.12
Input mode	Legacy Input Manager (activeInputHandler: 0)
Character model source	Assets/Characters/action.fbx; Generic rig import; Optimize Game Objects off
Animator state	Enabled; no Avatar and no Runtime Animator Controller assigned

1. Download and open the project

NOT CONFIRMED No repository URL, release page, or project README is present in the supplied workspace, so the original download source cannot be verified. Use the link or archive supplied by the project owner; do not substitute an unrelated repository.

SCENE RENAME The project owner confirms that Unity_Cardboard_Studio was renamed Unity_Adam_Studio. The inspected workspace snapshot still contains the earlier filename and Build Settings entry, so this tutorial uses the new owner-confirmed name while preserving the earlier filename in the evidence note at the end.

1. Download the project from the owner-provided source. If it arrives as a ZIP, extract it completely before opening Unity.
2. Install Unity Hub, then install Unity Editor 2022.3.62f1. Add WebGL Build Support only if you plan to make a browser build.
3. In Unity Hub, choose Add project from disk and select the folder FINAL_TOY_Mobile_V2 - WebGL. Select the folder that contains Assets, Packages, and ProjectSettings—not Assets itself.
4. Allow the first import to finish. Keep the existing Packages/manifest.json intact so Animation Rigging 1.2.1 and URP 14.0.12 resolve as authored.
5. Open Assets/Scenes/Unity_Adam_Studio.unity. Confirm that this renamed scene is enabled in File > Build Settings before making a new build.

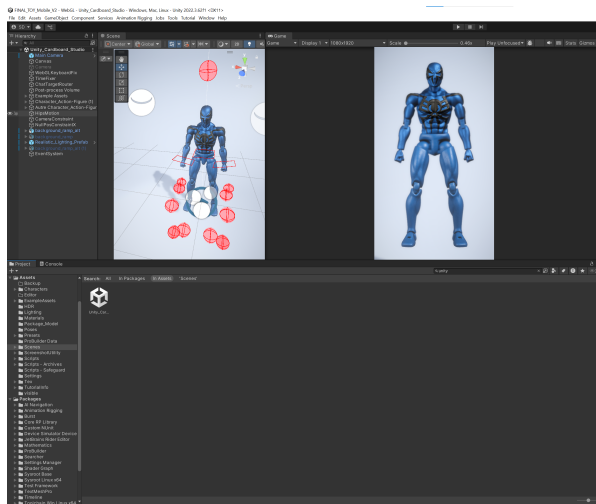


Figure 1. The correct scene is open, with Main Camera and Character_Action-Figure (1) visible in the Hierarchy.

Open the correct scene

1. In Project, open Assets > Scenes.
2. Double-click Unity_Adam_Studio.
3. Confirm the Hierarchy header also says Unity_Adam_Studio. The supplied screenshot was captured before the rename, so it may display the earlier scene name.

Run the packaged application without opening Unity

If the project download includes the completed Windows build and its shortcut, users do not need Unity Hub or the Unity Editor just to run the robot application.

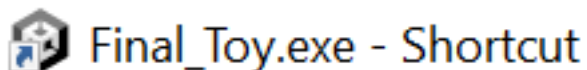


Final_Toy.exe - Shortcut

Launch screenshot A. The Unity-logo shortcut used to start Final_Toy.exe without opening the Unity Editor.

Launch the finished application

1. Locate the desktop or folder shortcut labeled Final_Toy.exe - Shortcut.
2. Double-click the shortcut. Windows follows its saved target and starts the packaged Unity application directly.
3. Keep the built executable and its accompanying build-data folder in their distributed locations. Moving only the executable can prevent a Unity build from starting.
4. If the shortcut reports that its target is missing, right-click it, open Properties, and inspect the Shortcut > Target field. The screenshots do not reveal the target path, so this document cannot confirm its exact location.



Launch screenshot B. Close view of the confirmed shortcut label: Final_Toy.exe - Shortcut.

EDITING VERSUS RUNNING Use the shortcut for experiencing and testing the already-built application. Open `Unity_Adam_Studio` in Unity only when you need to inspect the hierarchy, change the rig, modify controls, or create a new build.

2. Scene and hierarchy tour

The scene is a presentation studio around a single articulated robot. Top-level objects include Main Camera, Canvas, a second disabled Camera, WebGLKeyboardFix, TimeFixer, ChatTargetRouter, Post-process Volume, Example Assets, `Character_Action-Figure (1)`, a second character object, `HipsMotion`, `CameraConstraint`, `NullPosConstraintX`, background/lighting objects, and `EventSystem`.

IMPORTANT Main Camera is a prefab instance from `Assets/Scripts/Main Camera.prefab`. The separate `GameObject` named `Camera` has its `Camera` component disabled; it is not the primary view.

Read the scene before testing

The selected Main Camera prefab supplies the active Camera, `SimpleCameraController`, URP camera data, and `ActivateObject`.

The left Hierarchy exposes the imported bone tree and the red collider gizmos around interactive parts.

The Game view on the right is the runtime output; Scene view gizmos are editor-only.

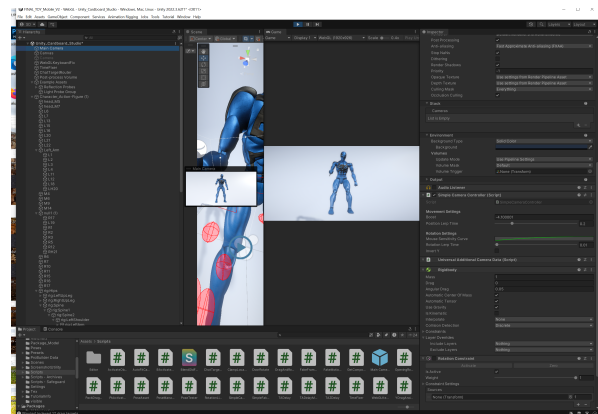


Figure 2. Main Camera selected, with the camera controller and physics/constraint components visible in the Inspector.



Figure 3. The project's imported Generic skeleton branch, including spine, neck, head, and arm bones.

Find the deforming skeleton

Expand `Character_Action-Figure (1)` > `rig:Hips` > `orig:Spine` > `rig:Spine1` > `rig:Spine2`.

The left arm continues through `rig:LeftShoulder`, `rig:LeftArm`, `mixamorig:LeftForeArm`, and `mixamorig:LeftHand`.

The neck/head branch is `rig:Neck` > `rig:Head`; the right side uses equivalent mixed `rig/mixamorig` names.

3. Test the supplied scene first

6. Enter Play Mode and wait for the Game view to settle.
7. Test both an editor portrait view (1080 x 1920) and the custom WebGL view (1920 x 926). The drag sensitivity is normalized against `Screen.width`, so resolution affects the feel.
8. Tap or left-click a visible robot part, then drag. Release the pointer and verify that the part stops rotating.
9. On desktop, hold the right mouse button to look around. Use W/A/S/D to move, Q/E to move vertically, Shift to boost, and the mouse wheel to change the movement boost.
10. Watch Console for [Router] Indexed 17 drag targets. That message is expected from `ChatTargetRouter` with the current cube-tagged set.

Portrait test profile

Choose the custom 1080x1920 Game view size.

Use this to evaluate one-finger interaction and how much of the robot remains reachable vertically.

The project uses a CanvasScaler reference resolution of 1080x1920, although the current interactive robot controls are physics objects, not UI buttons.

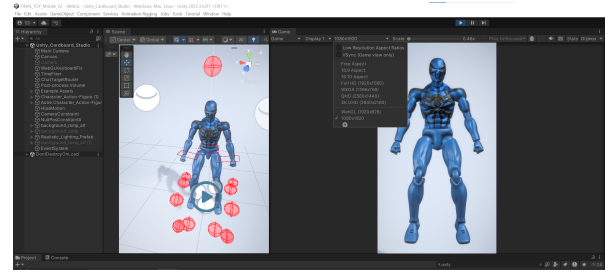


Figure 4. Portrait 1080x1920 selected in the Game view.

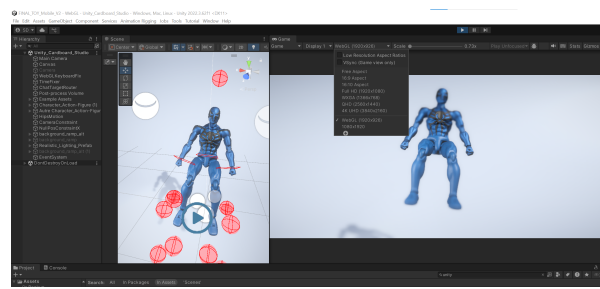


Figure 5. The custom WebGL 1920x926 Game view and the robot in a seated pose.

WebGL test profile

Choose WebGL (1920x926) to reproduce the authored browser-shaped Game view.

The script adds a WebGL-only sensitivity multiplier in a built browser player—not while running inside the Editor.

Compare this view with portrait before changing any per-part speed values.

Baseline appearance

Use this straight-on portrait pose as a quick visual baseline.

If the robot disappears, verify Main Camera is active and that Character_Action-Figure (1) is enabled.

If only gizmos disappear, remember that they are Scene view overlays and are not expected in Game view.

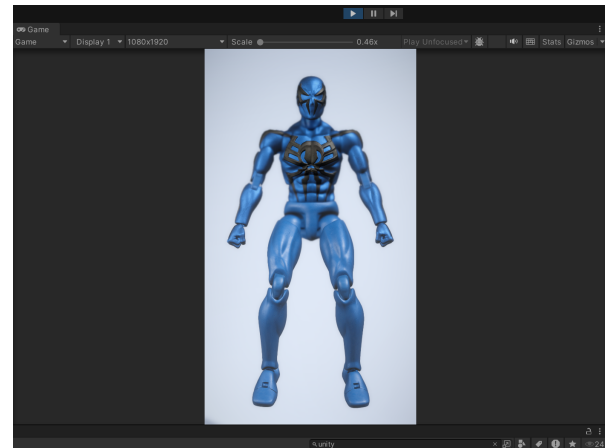


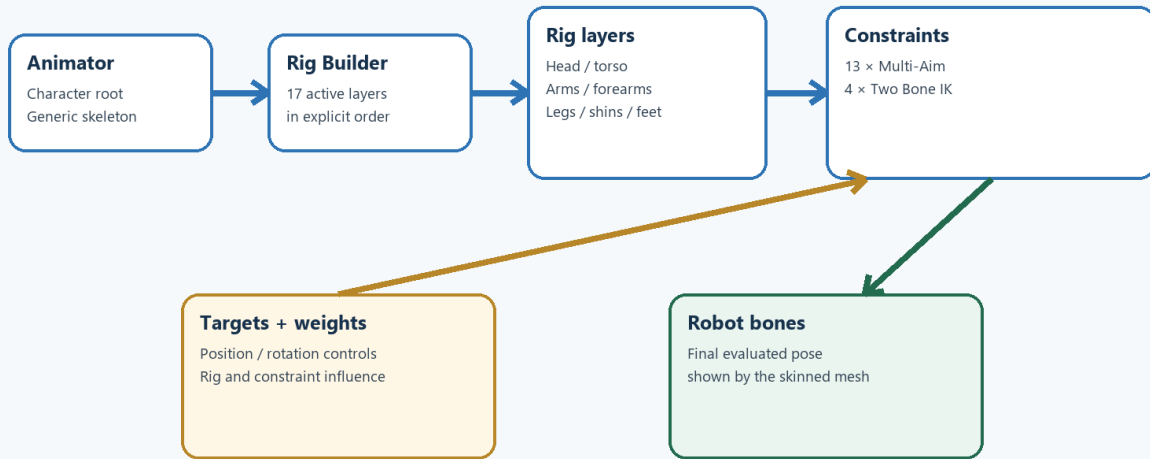
Figure 6. The clean runtime robot view without Scene gizmos.

4. How the existing robot is rigged

This project is a hybrid. It has a Generic bone hierarchy imported from action.fbx, an Animator and Rig Builder on Character_Action-Figure (1), a large Animation Rigging stack that drives bones from target transforms, and separate collider-bearing robot-part objects that the interaction scripts rotate directly.

How the robot rig evaluates

The skeleton is written by constraints; controls and targets remain easy to command.



Project note: pointer scripts also rotate some cube-tagged control objects directly; that is a parallel route.

Diagram A. Project-specific rig evaluation: Animator to Rig Builder, rig layers, constraints, targets, and bones.

Rig Builder layer order

#	Rig layer	Rig weight	Constraint inside
1	HeadRig (1)	0.4	Two Bone IK on NeckMover
2	HipsRig (1)	1.0	Two Bone IK on HipsMover
3	TorsoRig (1)	0.7	Multi-Aim, constraint weight 0.5
4	ArmRRig	1.0	Two Bone IK
5	ForearmRRig	0.7	Multi-Aim
6	BicepRRig	0.7	Multi-Aim
7	ShoulderRRig	0.7	Multi-Aim
8	FootRRig	0.7	Multi-Aim
9	LegRRig	0.7	Multi-Aim
10	ShinRRig	0.7	Multi-Aim
11	ArmLRig	1.0	Two Bone IK
12	ForearmLRig	0.7	Multi-Aim
13	BicepLRig	0.7	Multi-Aim
14	ShoulderLRig	0.7	Multi-Aim
15	FootLRig	0.7	Multi-Aim
16	LegLRig	0.7	Multi-Aim
17	ShinLRig	0.7	Multi-Aim

Two additional Rig Builder slots are serialized as None. All listed slots are active.

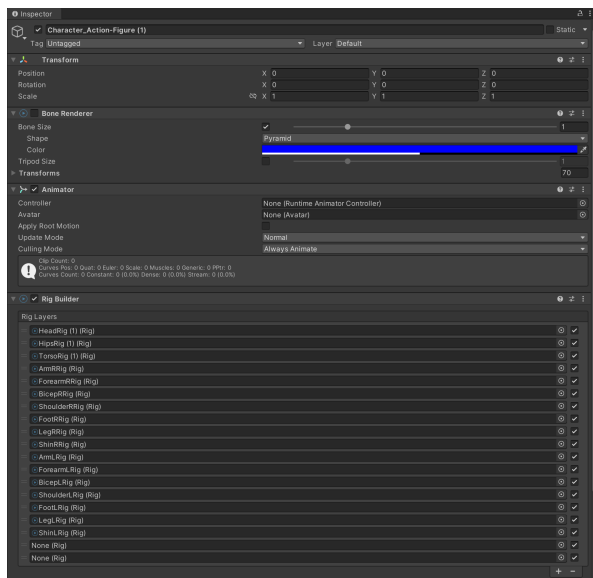


Figure 7. Character root with Animator and the long Rig Builder layer list.

Inspect the root rig stack

Select Character_Action-Figure (1).

Confirm Animator and Rig Builder are enabled.

The Animator intentionally has no controller and no avatar in this scene; Rig Builder still evaluates the transform hierarchy through its associated Animator.

Bone Renderer is present but disabled in the serialized scene; enable it only as an editor visualization aid.

Understand targets and controls

Each part rig contains a Target; many contain a child named HeadAim that holds the Multi-Aim Constraint.

The red spheres/capsules are collider gizmos for selectable robot pieces, not proof that every visible gizmo is an IK target.

ArmL/ArmR contain the strongest limb rigs; the other per-part rigs mainly aim individual bones toward targets.

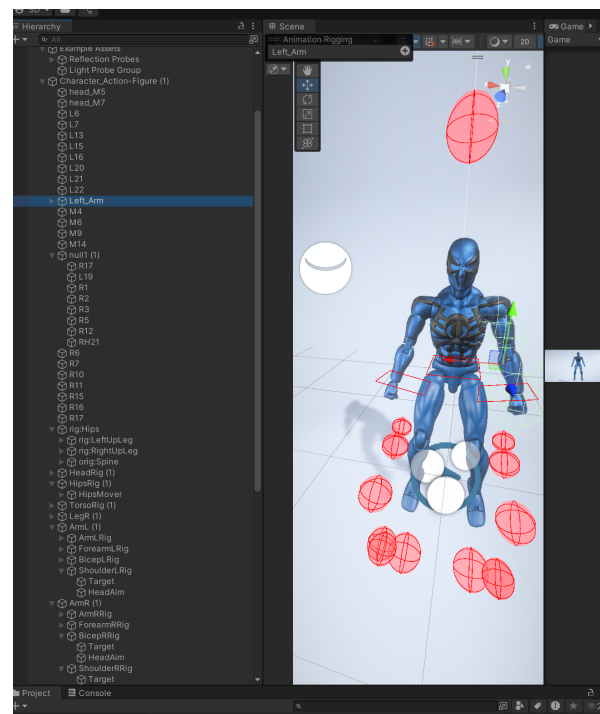


Figure 8. The full robot, collider gizmos, and per-part rig hierarchy.

Constraint details confirmed in the scene

- Thirteen Multi-Aim Constraints drive torso, shoulders, upper arms, forearms, thighs, shins, and feet from one weighted target each.
- Four Two Bone IK Constraints appear on the head, hips, left arm, and right arm. All have no Hint assigned.
- The arm IK target position weight is 1 and rotation weight is 0.4; head uses 0.7 rotation weight; hips uses 1.0.
- The torso Multi-Aim is limited to -15° through $+15^\circ$. Limb aim limits range from $\pm 30^\circ$ to $\pm 120^\circ$, with per-part offsets.

VERIFIED DEFECT TO INSPECT Both arm Two Bone IK constraints serialize Mid and Tip to the same hand transform instead of distinct forearm and hand transforms. The head and hips constraints also repeat a single transform for Root/Mid/Tip. That is not a valid textbook two-bone chain. Do not reproduce it when rigging a replacement robot; inspect it in the Editor before relying on IK behavior.

Two Bone IK: inspect the chain before copying it

The supplied scene contains repeated references; a replacement robot needs three distinct bones.

CURRENT SERIALIZED ARM

Verified in the analyzed studio scene

Root: upper arm

Mid: hand

Tip: same hand

X Mid and Tip resolve to one Transform

RECOMMENDED ROBOT ARM

Create this structure for stable IK

Root: upper arm

Mid: forearm

Target

Tip: hand

Hint controls bend

Diagram B. The serialized arm reference problem compared with a correct three-bone robot limb.

5. How desktop and mobile interaction works

The runtime path is simple and mostly independent of Animation Rigging:

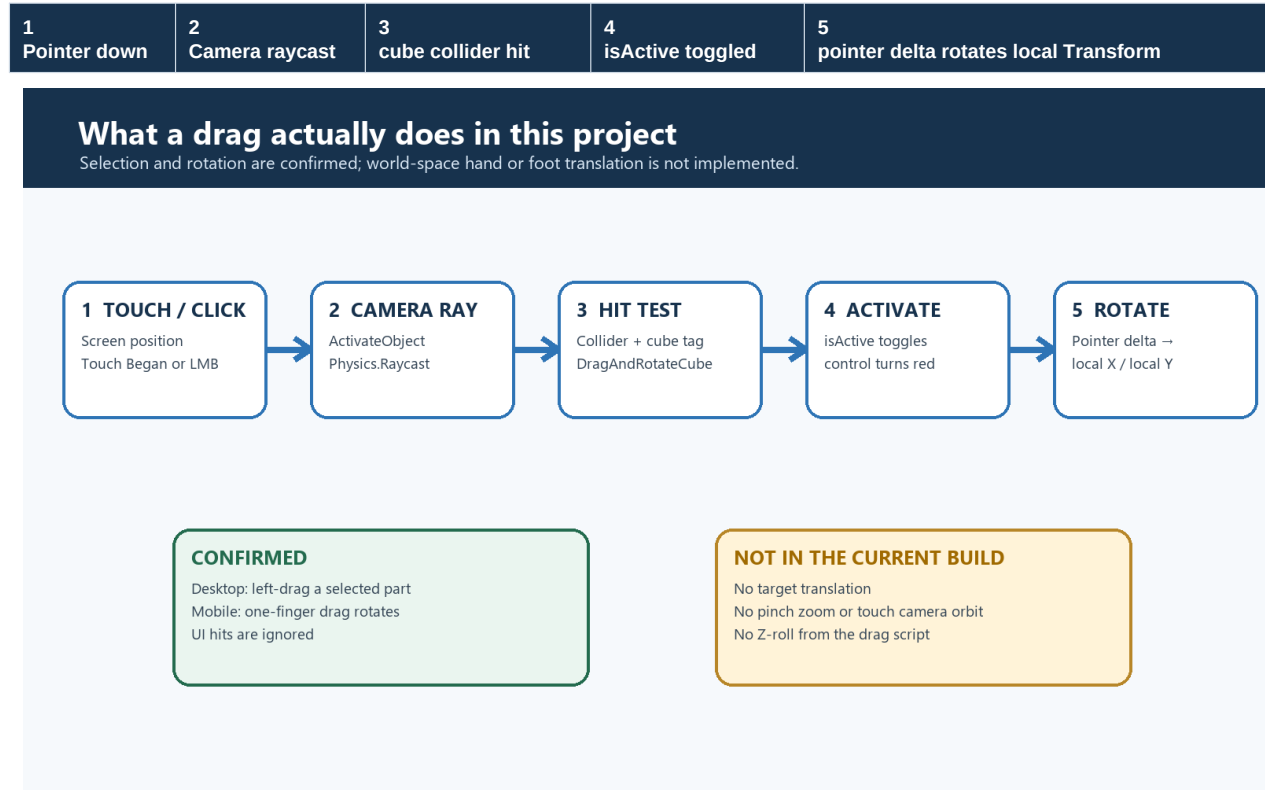


Diagram C. Confirmed pointer interaction path and the touch features that are not implemented.

Selection: ActivateObject

- Lives on Main Camera.prefab. On touch-began or left-mouse-down, it casts `Physics.Raycast(Camera.ScreenPointToRay(pointerPosition))`.
- It ignores a pointer that `EventSystem` reports over UI.
- It accepts all layers by default but requires the hit Transform to carry the cube tag.
- It looks for `DragAndRotateCube` on the hit collider and toggles `isActive`.

Rotation: DragAndRotateCube

- Requires a Collider. One-finger movement or held left-mouse movement produces a screen-space delta.
- Vertical drag becomes local X pitch; horizontal drag becomes local Y yaw; Z roll is intentionally zero.
- Effective speed = `rotateSpeedModifier × width normalization × partSpeedMultiplier`. A built WebGL player additionally uses `webglBoost × webglPartScale`.
- Touch Ended/Canceled and mouse-up deactivate the part. Active controls turn red; idle controls are gray when a `MeshRenderer` is present.
- `maxDegreesPerFrame` is zero on all confirmed instances, so no per-frame cap is active. Most tagged body parts use speed 0.45 or 0.65 and WebGL boost 2.2.

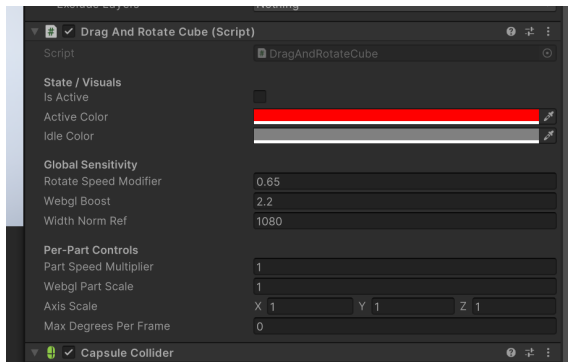


Figure 9. DragAndRotateCube and CapsuleCollider on an interactive part.

A selectable part

DragAndRotateCube is paired with a CapsuleCollider on many pieces.

The collider is what the camera ray hits; the script is what rotates the selected Transform.

Per-part sensitivity can soften or strengthen a control without changing the global algorithm.

Sensitivity fields

rotateSpeedModifier is the base pixel-to-degree multiplier.

widthNormRef defines the screen-width reference used by normalization.

axisScale can damp X or Y independently; the current script does not use Z for touch/mouse drag.

A zero maxDegreesPerFrame means uncapped rotation per frame.

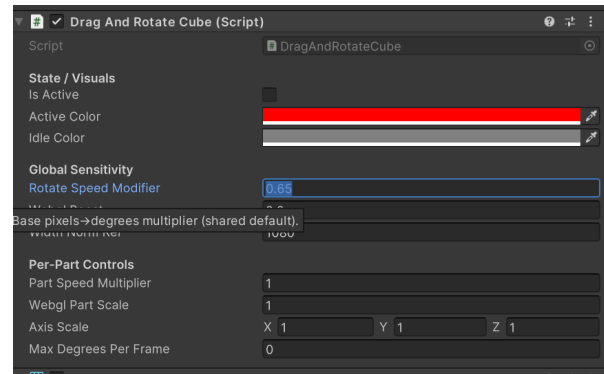


Figure 10. Close view of the DragAndRotateCube sensitivity and per-axis settings.

Desktop camera controls

- Right mouse drag rotates Main Camera and locks the cursor while held.
- W/A/S/D move forward/back/left/right; Q/E move vertically; Shift multiplies speed by 10; mouse wheel changes the exponential boost.
- These bindings come from SimpleCameraController. Because the project is set to legacy input and does not list the Input System package, the legacy Input branch is the confirmed active path.

Mobile controls and limitations

- Confirmed: one-finger tap selects a cube-tagged collider; one-finger movement rotates its Transform; finger release ends the interaction.
- Confirmed: EventSystem + StandaloneInputModule exist, and ActivateObject checks whether the pointer is over UI before selecting a robot part.
- Not implemented: touch translation of an IK target, pinch zoom, two-finger camera orbit, or touch camera movement.
- YDragAndRotateCube and BActivateObject exist in Assets/Scripts but are not attached in this scene. They are not part of the confirmed runtime path.

MEANING OF 'GRAB' AND 'MOVE' HERE The existing project supports grabbing in the sense of selecting a collider and moving the pointer to rotate/pose that object. It does not convert touch movement into world-space target translation. To let users drag hands or feet through space, add a separate positional-target driver to the appropriate IK Target; the current script cannot provide that behavior.

6. How scripts communicate with the rig

There are two parallel control routes, and conflating them would be misleading.

Route	What writes	What the rig receives
Human pointer	ActivateObject enables DragAndRotateCube; DragAndRotateCube writes the selected object's local rotation.	No direct RigBuilder or constraint API call. Most cube-tagged objects are separate mesh/control pieces, not the serialized IK Targets.
Chat/JS bridge	ChatTargetRouter indexes all cube-tagged transforms and accepts list/add/set/euler/resetIndex JSON commands.	It writes localRotation directly on the named transform. This is bridge-ready, but it bypasses constraint weights and target fields.
Animation Rigging	RigBuilder evaluates 17 active Rig layers after Animator evaluation.	Two Bone IK and Multi-Aim jobs read their target transforms and write the skeleton's constrained bones.

ChatTargetRouter API confirmed in code

- list — logs indexed part names.
- add or rotate — adds degrees on x, y, or z over an optional duration.
- set — sets one local Euler axis to an absolute degree value.
- euler — sets multiple local Euler values; unspecified axes are intended to remain unchanged.
- resetIndex — rebuilds the cube-tag index.

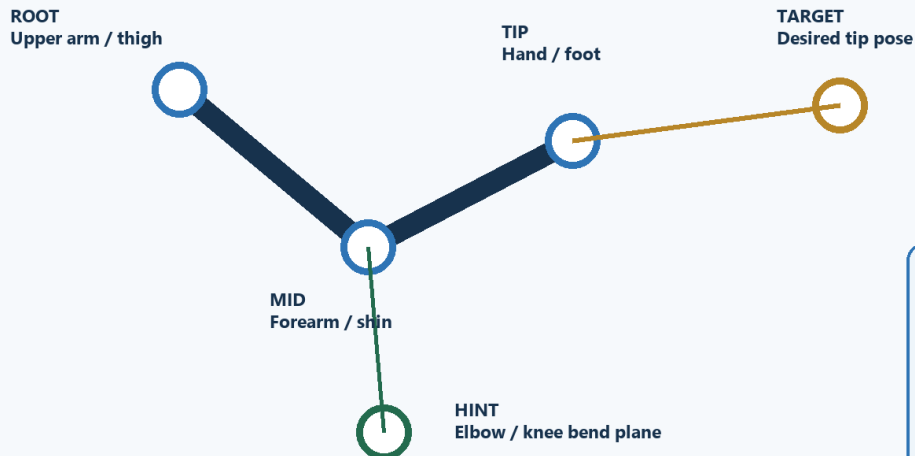
AI STATUS No AI model, network client, speech layer, or JavaScript sender is present in the Unity project. The preserved HandleJSON method is an integration endpoint, not a complete AI system. 'AI-ready' therefore means that a future trusted controller can emit validated, structured commands to stable transform names.

7. Rig a new robot character — video workflow adapted to this project

VIDEO WORKFLOW The referenced Brackeys video demonstrates a bone display, Rig Builder/Rig setup, Multi-Aim head/chest targets, separate arm rig layer, Two Bone IK, gizmo targets, Copy World Placement, target-offset choices, and runtime weight blending. The procedure below adapts those ideas to this project's robot and package versions.

Robot limb IK anatomy

Build the bone chain first, then place controls outside it.



Copy Target to Tip world placement first to avoid a snap; then move Target to pose the limb.

Diagram D. Two Bone IK anatomy for a robot arm or leg.

PHASE 01 / MODEL PREPARATION

Before Unity: prepare the robot

1. **In Blender, Maya, or another DCC, give the robot a real joint hierarchy.** For each two-joint limb, provide three distinct transforms: upper segment, lower segment, and end effector—for example UpperArm_L > Forearm_L > Hand_L.
2. **Skin flexible parts to bones, or parent rigid shell pieces cleanly to the appropriate bones.** Keep pivots on mechanical joint centers and orient local axes consistently.
3. **Export FBX with the skeleton and bind pose.** Unity Animation Rigging adds runtime controls; it does not create missing bones or skin weights.

PHASE 02 / UNITY IMPORT

Import and establish the Animator root

4. **Import the FBX under Assets.** In the Model Inspector > Rig tab, use Generic for a robot that does not need Humanoid retargeting. Apply the setting.
5. **Leave Optimize Game Objects off while building and debugging the rig so every required Transform remains addressable.** The supplied action.fbx is configured this way.
6. **Place the robot in a duplicate working scene.** Put an Animator on the character root. A controller is optional for procedural-only tests, but the Animator component itself is required by Rig Builder.

7. **Open Window > Package Manager and confirm Animation Rigging is installed.** For parity with this project, use package 1.2.1 with Unity 2022.3.62f1.

PHASE 03 / RIG FOUNDATION

Create the control-rig hierarchy

8. **Select the character root and use Animation Rigging > Rig Setup.** Verify the result: the Animator root receives Rig Builder and a child rig object receives Rig.
9. **Optionally use Animation Rigging > Bone Renderer Setup so bones are easier to inspect.** This changes editor visualization, not runtime deformation.
10. **Create stable control groups under the rig, such as HeadRig, TorsoRig, ArmLRig, ArmRRig, LegLRig, and LegRRig.** Use unique names because this project's command router indexes by GameObject name.

PHASE 04 / AIM CONSTRAINTS

Add head and torso aiming

11. **Under HeadRig, create HeadAim and add Multi-Aim Constraint.** Set Constrained Object to the head bone.
12. **Create a Target under HeadRig, place it in front of the robot, and add it as the first Source Object at weight 1.**
13. **Choose the head bone's true local forward axis as Aim Axis.** Set Up Axis and constrained axes, then set angular limits that fit the robot's neck stops.
14. **For a torso follow, create a second Multi-Aim Constraint on a chest/spine bone.** Reuse the same target if desired, but use a lower weight so the torso follows less than the head.

PHASE 05 / LIMB SOLVING

Add correct Two Bone IK limbs

15. **Under ArmLRig, create ArmLMover and add Two Bone IK Constraint.**
16. **Assign three different bones: Root = UpperArm_L, Mid = Forearm_L, Tip = Hand_L.** Mid must be a child of Root, and Tip must continue beneath Mid.
17. **Create Target and Hint controls.** Place Target at the hand's current world position and rotation; the video demonstrates copying the tip's world placement to avoid an immediate snap.
18. **Place Hint slightly in front of the elbow's desired bend plane and assign it.** Unlike the supplied scene's constraints, a production robot arm should normally use a real Hint to stabilize the elbow.
19. **Choose Target Position Weight and Target Rotation Weight.** Use Maintain Target Position/Rotation Offset only when you intentionally want to preserve the current difference between tip and target.
20. **Repeat for the right arm and both legs.** For legs use Thigh > Shin > Foot; place knee hints forward and keep foot targets outside the deforming bone chain.
21. **Add each Rig object to Rig Builder in evaluation order.** Put broader body adjustments before or after limb solvers deliberately, then test; order changes the result when several constraints write related bones.

PHASE 06 / RUNTIME CONTROLS

Make controls interactive in this project

22. **For rotational controls, add a visible or invisible collider to the intended control object, tag that same GameObject cube, and add DragAndRotateCube.** The script checks the hit Transform itself; a tag only on a parent will not satisfy CompareTag on a child collider.
23. **Tune rotateSpeedModifier, widthNormRef, axisScale, and WebGL multipliers per control.** Add ClampLocalRotation or a mechanical-limit component when a joint must stop safely.
24. **For hand/foot positional dragging, create a new position driver that raycasts onto a plane or depth surface and moves the IK Target in world space.** Do not expect DragAndRotateCube to translate it.
25. **Scope ChatTargetRouter.root to the new character and expose stable, unique target names.** Prefer commanding IK targets and Rig/constraint weights over directly writing deforming bones.
26. **Validate every incoming AI command: allowed target names, axis ranges, duration limits, and joint limits.** Keep the command layer deterministic; an AI should request motion, while Unity enforces what is mechanically legal.

8. Replace the existing robot

11. Duplicate Unity_Adam_Studio.unity before editing. Keep the original scene as a working reference.
12. Import and place the new robot under a clean root with Animator. Match the old character's world position and scale only after confirming the new model's forward axis and units.
13. Build a fresh rig with the procedure in Section 7. Do not drag the old scene's bone references into new constraints; every Root/Mid/Tip/Constrained Object must belong to the new Animator hierarchy.
14. Recreate targets, hints, colliders, cube tags, and interaction scripts. Copy numerical tuning only after testing—the same pixel delta can feel very different on a differently sized robot.
15. Update ChatTargetRouter.root to the new root and verify all names are unique. The current router keeps the first duplicate name it sees, so duplicates can route commands unpredictably.
16. Disable the old Character_Action-Figure (1) only after the new robot passes desktop, portrait, and WebGL tests. Then update camera framing and post-processing as needed.

9. Test checklist

- No Console compile errors, missing scripts, or PropertyStreamHandle errors.
- Rig Builder has the intended Rig layers, no accidental None slots, and the correct order.
- Every Two Bone IK uses distinct Root, Mid, and Tip transforms and every chain bends in the intended plane.
- Targets begin at sensible positions; entering Play Mode does not snap limbs unexpectedly.
- Head/torso aim uses the correct local aim axis and respects angular limits.
- Every selectable collider is on the same GameObject that has cube tag and DragAndRotateCube.
- One touch selects and rotates exactly one part; releasing stops it; UI taps do not select the robot.
- Desktop camera controls do not interfere with left-drag posing.
- 1080x1920 and 1920x926 both feel controllable; a built WebGL player is tested separately from the Editor.
- ChatTargetRouter lists the expected unique names and can execute a small, clamped test command.

10. Common problems and fixes

Symptom	Fix
A limb does not solve	Check that Root, Mid, and Tip are three distinct transforms in one chain. The supplied scene's arm Mid/Tip duplication is a concrete example of what to inspect.
PropertyStreamHandle cannot be resolved	Keep rigs and targets under the Animator hierarchy; disable Optimize Game Objects while debugging; rebuild Rig Builder after hierarchy changes.
The limb snaps on Play	Copy the tip's world position/rotation to the target before enabling the constraint, or use Maintain Target Offset intentionally.
Elbow or knee flips	Assign a Hint and put it clearly on the desired bend side. The supplied scene has no hints, so it is more vulnerable to ambiguous bends.
Head aims sideways	Correct Aim Axis and Up Axis for the imported bone's local orientation; do not assume world Z is local forward.
Tap does nothing	Verify Main Camera is active, the collider is hit by its culling/layer setup, the hit GameObject has tag cube, and DragAndRotateCube is on that same object.
Drag rotates too fast in WebGL	Lower per-part rotateSpeedModifier or webglPartScale; remember the WebGL boost compiles only in a non-Editor WebGL player.
Part spins too far	Set maxDegreesPerFrame and add joint clamps. Current tagged controls mostly have no cap, so this protection is not active by default.
Touch cannot move a hand target	That is expected: no positional touch driver exists. Implement a plane/depth-based target translator and move the IK Target, not the mesh shell.
AI command moves the wrong part	Use unique names, set ChatTargetRouter.root, rebuild the index, and reject unknown targets. The router is case-insensitive but keeps only the first duplicate name.
Camera does not respond on mobile	Also expected: SimpleCameraController exposes mouse/keyboard/gamepad navigation, not touch orbit or pinch zoom.
Rig changes are invisible in Animation preview	Confirm Rig Builder and Rig are enabled and use the Animation Rigging preview/inverse tools as appropriate; also verify constraint weight and Rig weight are above zero.

11. What is confirmed—and what is not

Confirmed

- Unity version, main scene path, package versions, build scene, URP, legacy input mode, tags, and relevant model-import settings.
- The root Animator/Rig Builder, 17 active non-null Rig layers, 13 Multi-Aim Constraints, four Two Bone IK Constraints, targets, weights, and serialized chain references.
- Main Camera prefab components, legacy desktop camera bindings, Physics.Raycast selection, cube tag filter, collider requirement, touch/mouse rotation mapping, WebGL sensitivity math, and router JSON commands.
- All twelve supplied screenshots and their visible Inspector/Hierarchy/Game-view/shortcut evidence.

Not confirmed / not present

- The project's original public download URL or license, because no repository metadata or README was supplied.
- A completed AI service integration; only a local JSON command endpoint exists.
- Touch-based world-space translation, pinch zoom, or two-finger orbit; the project code does not implement them.

- Whether the degenerate Two Bone IK assignments are deliberate workarounds or mistakes; the serialized references are confirmed, but intent is not.

Sources

Project evidence (read-only)

- ProjectSettings/ProjectVersion.txt
- Packages/manifest.json
- Packages/packages-lock.json
- ProjectSettings/EditorBuildSettings.asset
- ProjectSettings/ProjectSettings.asset
- ProjectSettings/TagManager.asset
- Assets/Scenes/Unity_Adam_Studio.unity (owner-confirmed current name; inspected snapshot: Assets/Scenes/Unity_Cardboard_Studio.unity)
- Assets/Scripts/Main Camera.prefab
- Assets/Scripts/ActivateObject.cs
- Assets/Scripts/DragAndRotateCube.cs
- Assets/Scripts/ChatTargetRouter.cs
- Assets/Scripts/SimpleCameraController.cs
- Assets/Scripts/ClampLocalRotation.cs
- Assets/Scripts/AutoFitCapsule.cs
- Assets/Characters/action.fbx.meta

External technical references

Brackeys — Make your Characters Interactive! - Animation Rigging in Unity

Unity 2022.3 manual — Animation Rigging package availability

Unity Animation Rigging 1.2 — RigBuilder API

Unity Animation Rigging 1.2 — Two Bone IK manual

Unity Animation Rigging 1.2 — Multi-Aim Constraint manual